

CSS Layout – Flexbox

Introduktion

Flexbox är CSS-systemet som hanterar **endimensionella layouter**: antingen en **rad** eller en **kolumn**. Det är byggt för att lösa problem som tidigare krävde hack med floats, inline-block eller absolut positionering.

Flexbox är logiskt, förutsägbart och perfekt för komponenter som ska:

- ligga på en rad
- centreras
- fördelas jämnt
- anpassa sig efter innehållets storlek
- wrap:a när utrymmet tar slut

Flexbox är alltså ett **content-first** system: innehållet styr layouten.

1. Flex Container & Flex Items

När du skriver:

```
display: flex;
```

på ett element blir det en **Flex Container**. Alla dess direkta childs blir **Flex Items**.

Exempel:

```
<div class="menu">
  <button>Start</button>
  <button>Produkter</button>
  <button>Kontakt</button>
</div>

.menu {
  display: flex;
}
```

Nu ligger knapparna på en rad.

2. Axelsystemet (Main Axis & Cross Axis)

Flexbox styrs av två axlar:

Main Axis (huvudaxeln)

Det är den axel elementen placeras längs.

Cross Axis (korsaxeln)

Det är den axel elementen linjeras längs.

Vilken axel som är vilken beror på `flex-direction`.

3. flex-direction

Bestämmer riktningen på huvudaxeln.

```
flex-direction: row;      /* Standard: vänster → höger */
flex-direction: column;   /* Uppifrån → ner */
```

Effekt:

- `row` → elementen ligger horisontellt
- `column` → elementen ligger vertikalt

4. justify-content (Main Axis)

Styr hur elementen fördelas längs huvudaxeln.

```
justify-content: flex-start; /* Standard */
justify-content: center;
justify-content: flex-end;
justify-content: space-between;
justify-content: space-around;
justify-content: space-evenly;
```

Exempel: centrera en rad med knappar:

```
.menu {
  display: flex;
  justify-content: center;
}
```

5. align-items (Cross Axis)

Styr hur elementen linjeras längs korsaxeln.

```
align-items: flex-start;
align-items: center;
align-items: flex-end;
stretch; /* Standard */
```

Exempel: centrera i höjddled:

```
.menu {
  display: flex;
  align-items: center;
}
```

6. flex-wrap

Bestämmer vad som händer när elementen inte får plats.

```
flex-wrap: nowrap; /* Standard */
flex-wrap: wrap;
flex-wrap: wrap-reverse;
```

Exempel: låt kort hoppa ner på ny rad:

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}
```

Enheterna px och rem

px – Pixel (absolut enhet)

En fast enhet som inte skalar med skärm eller textinställningar.

Bra för exakt kontroll, ikoner och tunna linjer.

Exempel:

```
width: 200px
```

rem – Root-EM (responsiv enhet)

Relativ till html-elementets font-size (standard 16px). Skalar automatiskt när användaren ökar textstorlek → tillgängligt och responsivt.

Exempel:

```
font-size: 1.5rem;
```

$1.5 \times 16\text{px} = 24\text{px}$ (men skalar om root-storleken ändras).

7. gap

Sätter avstånd mellan Flex Items utan margin-hack.

```
gap: 1rem;
```

Detta är en av de mest praktiska egenskaperna i modern CSS.

8. flex (Children)

Flexbox blir riktigt kraftfullt när du stylar underliggande childs.

```
.item {  
  flex: 1; /* Tar lika mycket plats */  
}
```

Eller:

```
.item {  
  flex: 2; /* Tar dubbelt så mycket plats som flex:1 */  
}
```

Flex är en shorthand för:

```
flex-grow flex-shrink flex-basis
```

Exempel:

```
.item {  
  flex: 1 1 200px;  
}
```

9. När ska man använda Flexbox?

Flexbox är bäst för:

- Navbars
- Footers
- Knappar
- Formulär
- Kortlistor
- Taggar / chips
- Produktkort som ska wrap:a
- Vertikal centering
- Horisontell centering
- Små komponenter

Flexbox är **inte ersatt av Grid** — de kompletterar varandra.